

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x01 实验报告

操作系统的启动

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 编译内核 ELF

在 `crates/kernel` 目录下运行 `cargo build --release` 命令后，在 `target/x86_64-unknown-none/release` 目录下找到 `ysos_kernel` 文件，使用 `readelf` 命令查看，得到以下内容：

```
$ readelf -h ysos_kernel
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              EXEC (Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0xffffff00000010d0
  Start of program headers:          64 (bytes into file)
  Start of section headers:          49712 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         7
  Size of section headers:           64 (bytes)
  Number of section headers:         12
  Section header string table index: 10
```

对比配置文件可知，该编译产物的架构相关信息与其中的描述一致，如 `Data` 中的 `little endian` 对应配置文件中的 `"target-endian": "little"`。

根据上述信息，内核的入口点（Entry point address）为 `0xffffff00000010d0`。根据 `kernel.ld` 中的信息，内核的基址设置为 `KERNEL_BEGIN = 0xffffff0000000000;`，之所以这个地址和入口点不同是因为在加载程序之前，会使用 `kernel.ld` 脚本来进行链接，会链接 `.rodata` 等放置在头部（这部分信息是只读的），根据下述的信息，第一个 `LOAD` 的信息类型是只读，而第二个就是可执行，第一部分占有的空间是 `0x0d84`，偏置量为 `0x1000`，从而得到内核的入口点 `0xffffff00000010d0`。

```
$ readelf -l ysos_kernel

Elf file type is EXEC (Executable file)
Entry point 0xffffff00000010d0
There are 7 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  LOAD           0x0000000000001000 0xffffff0000000000 0xffffff0000000000
                 0x0000000000000d84 0x0000000000000d84  R      0x1000
  LOAD           0x0000000000002000 0xffffff0000001000 0xffffff0000001000
                 0x00000000000035e4 0x00000000000035e4  R E    0x1000
  LOAD           0x0000000000006000 0xffffff0000005000 0xffffff0000005000
                 0x0000000000001020 0x0000000000001020  RW     0x1000
  LOAD           0x0000000000008000 0xffffff0000007000 0xffffff0000007000
                 0x0000000000000000 0x0000000000000030  RW     0x1000
  GNU_RELRO      0x0000000000007000 0xffffff0000006000 0xffffff0000006000
                 0x0000000000000020 0x0000000000000020  R      0x1
  GNU_EH_FRAME   0x0000000000001d0c 0xffffff0000000d0c 0xffffff0000000d0c
                 0x000000000000001c 0x000000000000001c  R      0x4
  GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x0000000000000000 0x0000000000000000  RW     0x0

 Section to Segment mapping:
  Segment Sections...
   00     .rodata .eh_frame_hdr .eh_frame
   01     .text
   02     .data .got
   03     .bss
   04     .got
   05     .eh_frame_hdr
   06
```

根据上述的信息，得知编译产物共有七个 segments，具体信息整理如下：

段编号	类型	权限	对齐要求	包含的主要节区	备注
00	`LOAD`	只读 (R)	`0x1000`	`.rodata` 等	只读数据段，按 4KB 对齐
01	`LOAD`	读/执行 (R E)	`0x1000`	`.text`	代码段，按 4KB 对齐
02	`LOAD`	读/写 (RW)	`0x1000`	`.data .got`	数据段，按 4KB 对齐
03	`LOAD`	读/写 (RW)	`0x1000`	`.bss`	BSS 段（文件大小为 0，内存中占用空间），按 4KB 对齐

04	`GNU_RELRO`	只读 (R)	`0x1`	`.got`	只读重定位区域，用于保护（通常与 `LOAD` 段合并）
05	`GNU_EH_FRAME`	只读 (R)	`0x4`	`.eh_frame_hdr`	异常处理帧头，按 4B 对齐
06	`GNU_STACK`	读/写 (RW)	`0x0`	无	栈段标记，实际不占用文件/内存空间

2. 在 UEFI 中加载内核

2.1. 加载相关文件

根据文档中给出的文件读取示例，观察 `config.rs` 中 `config` 类型的定义和实现以及 `ElfFile` 类型的定义和实现来调用相关的函数在加载配置文件以及内核 ELF，代码如下：

```
// 1. Load config
    let config = {
        let mut file = open_file(CONFIG_PATH);
        let buf = load_file(&mut file);
        Config::parse(buf)
        // FIXED
    };

// 2. Load ELF files
    let elf = {
        let mut file = open_file(config.kernel_path);
        let buf = load_file(&mut file);
        ElfFile::new(buf).expect("Invalid ELF file")
        // FIXED
    };
```

2.2. 更新控制寄存器

由于我们需要修改根页表，因此需要先禁用 `CR0` 的写保护，在修改之后，再还原 `CR0` 的写保护，禁用和还原的代码如下：

```
// FIXED: root page table is readonly, disable write protect (Cr0)
    unsafe {
        Cr0::update(|flags| {
            flags.remove(Cr0Flags::WRITE_PROTECT);
        });
    }
// FIXED: recover write protect (Cr0)
    unsafe {
        Cr0::update(|flags| {
            flags.insert(Cr0Flags::WRITE_PROTECT);
        });
    }
```

2.3. 映射内核文件

我们首先通过 `map_physical_memory` 函数将物理地址映射到虚拟地址，然后通过 `load_elf` 函数加载 ELF 文件的内容到对应的地址，最后用 `map_range` 函数分配内核的栈空间，观察每个函数需要传入的参数进行调用即可：

```
// FIXED: map physical memory to specific virtual address offset
    let mut frame_allocator = UEFIFrameAllocator;

    map_physical_memory(
        config.physical_memory_offset,
        max_phys_addr,
        &mut page_table,
        &mut frame_allocator,
    );
// FIXED: load and map the kernel elf file
    load_elf(
        &elf,
        config.physical_memory_offset,
        &mut page_table,   // load_elf expects Mapper<Size4KiB>
        &mut frame_allocator,
    )
    .expect("Failed to load ELF");
// FIXED: map kernel stack
    map_range(
        config.kernel_stack_address,
        config.kernel_stack_size, // number of 4KiB pages
        &mut page_table,
        &mut frame_allocator,
    )
```

这里需要补全 `load_segment` 函数的实现，具体而言就是要根据 `segment` 的标记位更新页表的标记位，补全的代码如下：

```rust
// FIXED: handle page table flags with segment flags
// unimplemented!("Handle page table flags with segment flags!");
    if segment.flags().is_write() {
        page_table_flags |= PageTableFlags::WRITABLE;
    }
    if !segment.flags().is_execute() {
        page_table_flags |= PageTableFlags::NO_EXECUTE;
    }
    if segment.flags().is_read() {
        page_table_flags |= PageTableFlags::USER_ACCESSIBLE;
    }
```

2.4. 实验任务

2.4.1. `set_entry` 函数做了什么？为什么它是 `unsafe` 的?

`set_entry` 函数设置了内核的入口点地址。由于 `set_entry` 函数内部存在 `unsafe` 块，且它写入的变量 `ENTRY` 是 `static mut` 类型，该类型的读写都需要 `unsafe` 进行标记。(且写入的地址无法由编译器验证，原则上需要设为 `unsafe`)

2.4.2. `jump_to_entry` 函数做了什么？要传递给内核的参数位于哪里？查询 `call` 指令的行为和 x86_64 架构的调用约定，借助调试器进行说明。

`jump_to_entry` 将控制权转移给内核，具体而言就是设置好内核的栈顶，并跳转到内核的入口点，开始运行内核。根据该函数内部调用的汇编指令，要传递给内核的信息 `bootinfo` 存在内存中，使用传递地址的方式传递给内核；按照 x86-64 System V 调用约定，函数调用的前六个参数依次通过以下寄存器传递：`RDI`，`RSI`，`RDX`，`RCX`，`R8`，`R9`，而 `&bootinfo` 就是传递给内核的第一个参数，因此也可以说明其是通过 `RDI` 寄存器传递的。我们也可以借助 gdb 的 `print` 进行观察：

```
gef➤  print boot_info
$1 = (*mut ysos_boot::BootInfo) 0x5f0deb0
gef➤  print $rdi
$2 = 0x5f0deb0
```

从上图中我们可以看到 `boot_info` 的值(对应于 `&bootinfo`)和 `RDI` 寄存器中的值是一样的。

2.4.3. `entry_point!` 宏做了什么？内核为什么需要使用它声明自己的入口点?

观察 `entry_point!` 展开的代码，它创建了一个 `_start` 函数，用于作为内核的入口点。这个宏相当于是作为一个统一的内核入口接口的作用，这样声明可以保证默认入口的定义，类型，签名都符合链接器寻找内核入口点的标准，同时可以在编译期检查类型正确，以及简化真实内核入口函数的实现。

2.4.4. 如何为内核提供直接访问物理内存的能力？你知道几种方式？代码中所采用的是哪一种？

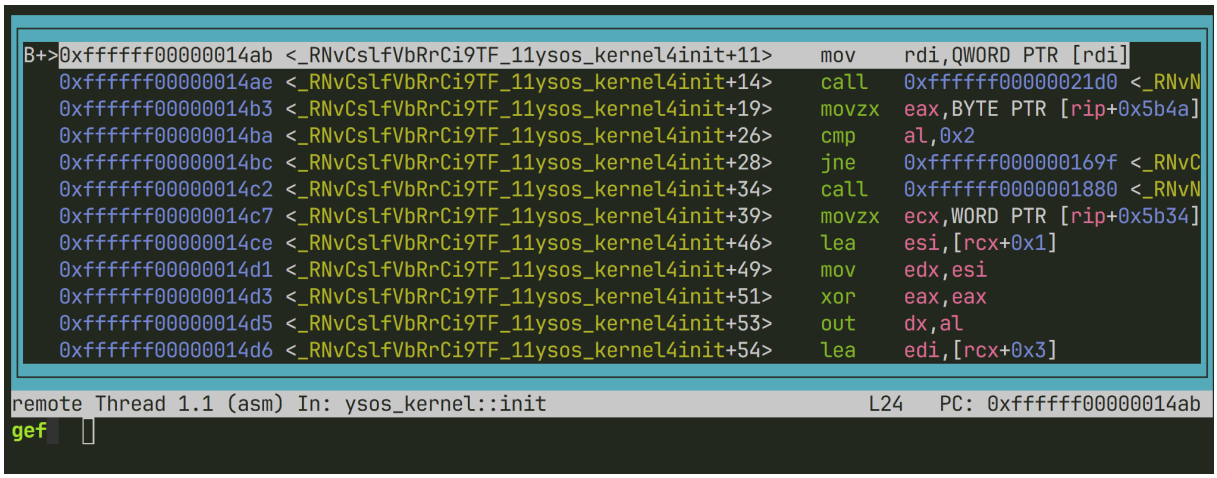
查阅资料，主要有三种方式：恒等映射（物理地址就是虚拟地址）；线性映射（虚拟地址和物理地址之间相差一个固定的偏移量）；动态映射（不预先映射所有的物理地址，当内核需要访问某块特定的物理内存时，会在内核的虚拟地址空间中临时分配一段地址）。本实验代码采用的是第二种，通过设置偏移量把物理地址映射到虚拟地址。

2.4.5. 为什么 ELF 文件中不描述栈的相关内容？栈是如何被初始化的？它可以被任意放置吗？

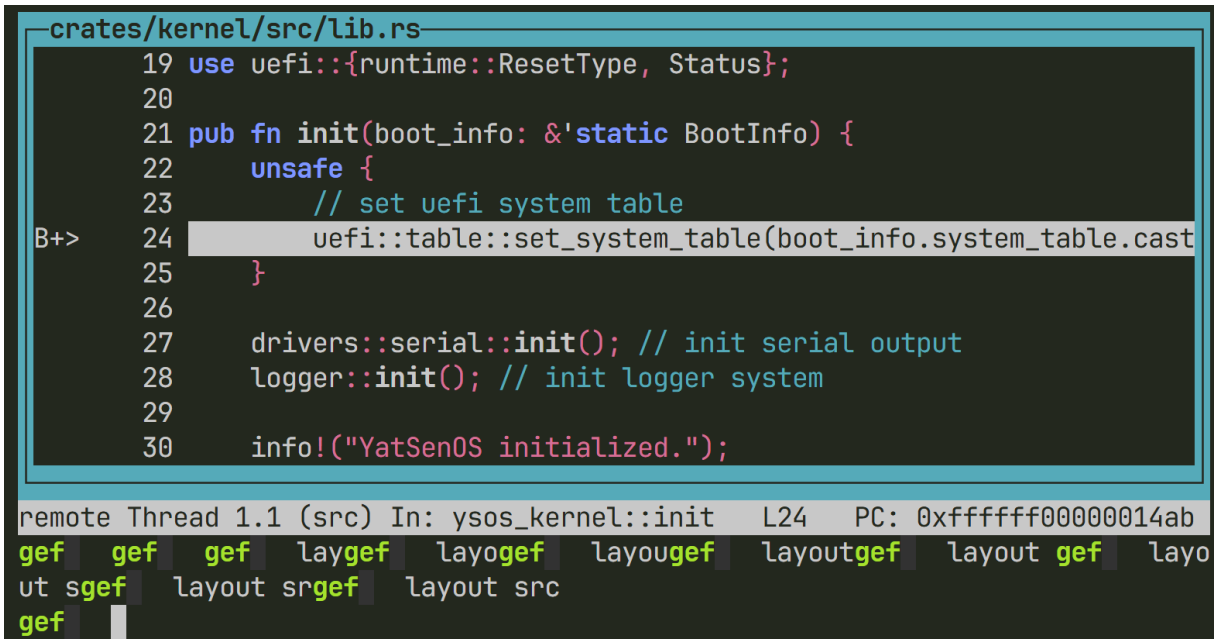
因为栈的大小是动态变化的，一般由操作系统进行栈的空间分配和初始化，不同的操作系统对应栈的布局可能有所不同，因此栈的相关内容不是编译时能确定的，不会在 ELF 文件中进行描述的。根据本实验的代码，首先在建立页表映射时调用 `map_range()` 分配栈空间，随后在跳转到内核入口时，由 `boot` 分配栈顶指针来进行初始化的。理论上，只要不与其他数据所在的区域冲突，且空间大小和地址（如需栈指针对齐）满足操作系统的栈的需求，则可以作为栈所在的空间。

2.4.6. 请解释指令 `layout asm` 的功能。倘若想找到当前运行内核所对应的 Rust 源码，应该使用什么 GDB 指令？

`layout asm` 会展示内核运行的汇编源码，会高亮标出下一条即将执行的指令，如下图：

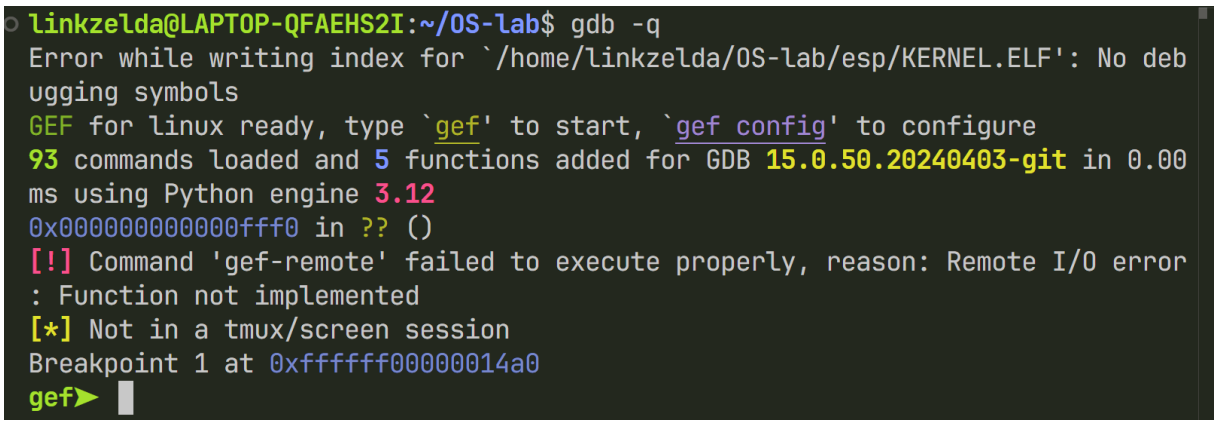


如果想类似的找到当前运行的 Rust 源码，可以用 `layout src` 指令，如下图：



2.4.7. 假如在编译时没有启用 `DBG_INFO=true`，调试过程会有什么不同?

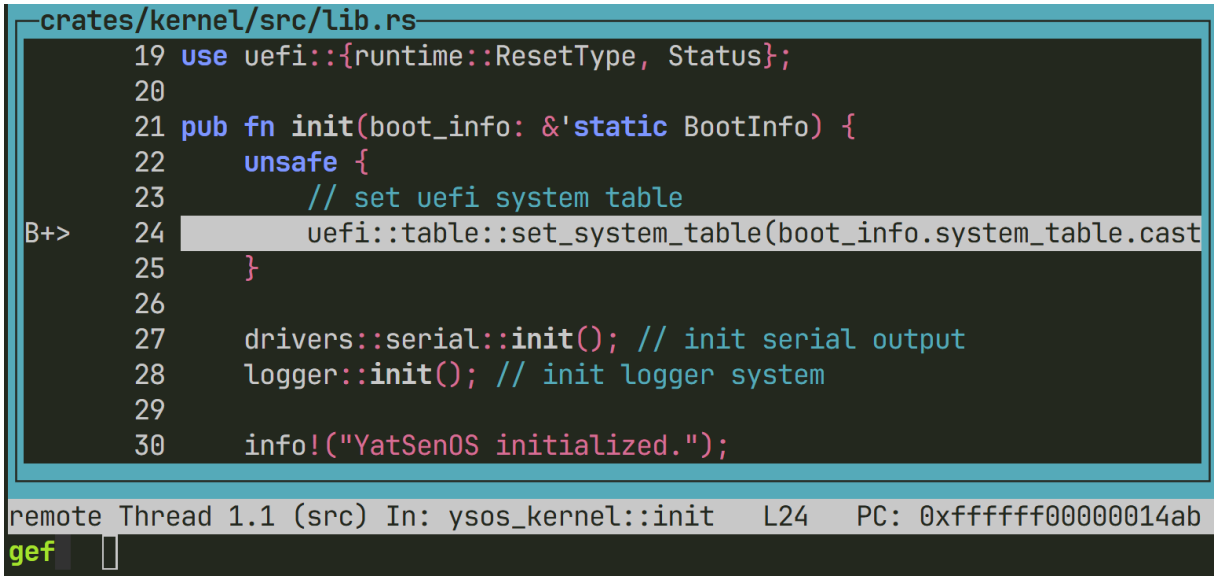
如果编译时没有启用 `DBG_INFO=true` 的话，就不能基于源码进行调试（例如不能在源码行上设置断点，断点显示不会自动映射到源码行上等），只可以基于汇编指令进行调试（可以使用内存地址查看数据内容）。如下图（图一不会显示断点对应的代码行，图二启用了 `DBG_INFO=true`，会给出对应的代码行）：





2.4.8. 你如何选择了你的调试环境? 截图说明你在调试界面（TUI 或 GUI）上可以获取到哪些信息?

我采用了使用 WSL 的 GDB 结合 GEF 插件进行调试，可以使用 `layout src` 命令打开 TUI 调试界面，可以获取当前运行到的程序信息，会高亮展示下一条即将执行的语句，下方会给出当前的线程编号，执行的函数名，以及 PC 寄存器的值。



3. UART 与日志输出

3.1. 串口驱动的实现

笔者根据文档中的实例进行实现，且在代码中加入了相应的注释，代码如下：

3.1.1. 初始化

```
/// Initializes the serial port.
    pub fn init(&self) {
        // FIXED: Initialize the serial port
        let base = self.base;

        // 禁用所有中断（IER 寄存器写 0）
        unsafe {
            Port::new(base + IER).write(0x00u8);
        }

        // 设置 DLAB = 1 以访问除数锁存器
        unsafe {
            Port::new(base + LCR).write(0x80u8);
        }

        // 设置波特率除数
        unsafe {
            Port::new(base + DATA).write(0x03u8); // 低字节
            Port::new(base + IER).write(0x00u8);    // 高字节
        }

        // 设置数据格式：8 位数据，无校验，1 位停止位
        unsafe {
            Port::new(base + LCR).write(0x03u8);
        }

        // 启用 FIFO，清除 FIFO，设置 14 字节阈值
        unsafe {
            Port::new(base + FCR).write(0xC7u8);
        }

        // 先设置 MODEM 控制寄存器为 0x0B，再设为 0x1E 进入回环模式
        unsafe {
            Port::new(base + MCR).write(0x0Bu8);
            Port::new(base + MCR).write(0x1Eu8);
        }

        // 测试芯片：写入 0xAE，检查读入数据是否相同
        unsafe {
            Port::new(base + DATA).write(0xAEu8);
            let test_byte: u8 = Port::new(base + DATA).read();
            if test_byte != 0xAE {
                panic!("Serial port test failed: sent 0xAE, received 0x{:X}",
test_byte);
            }
        }

        // 设置正常操作模式：回环关闭，IRQ 启用，OUT1/OUT2 置位
        unsafe {
            Port::new(base + MCR).write(0x0Fu8);
        }
    }
```

3.1.2. 发送数据

```
/// 判断发送保持寄存器是否为空
    pub fn is_transmit_empty(&mut self) -> bool {
        let base = self.base;

        let lsr: u8 = unsafe { Port::new(base + LSR).read() };
        (lsr & 0x20) == 0
    }

/// Sends a byte on the serial port.
    pub fn send(&mut self, data: u8) {
        // FIXED: Send a byte on the serial port
        let base = self.base;
        // 等待发送保持寄存器为空
        while self.is_transmit_empty() {
            core::hint::spin_loop();
        }
        // 写入数据寄存器
        unsafe { Port::new(base + DATA).write(data); }
    }
```

3.1.3. 接收数据（不等待）

```
/// Receives a byte on the serial port no wait.
    pub fn receive(&mut self) -> Option<u8> {
        // FIXED: Receive a byte on the serial port no wait
        let base = self.base;
        let lsr: u8 = unsafe { Port::new(base + LSR).read() };
        if (lsr & 0x01) != 0 {
            // 若数据就绪，则读入并返回
            let data = unsafe { Port::new(base + DATA).read() };
            Some(data)
        } else {
            None
        }
    }
```

3.2. 日志输出

根据 log crate 提供的函数进行实现，为了方便调试，这里设置了日志的输出级别为 Trace，具体的关键代码如下：

```
// FIXED: Configure the logger
    log::set_max_level(LevelFilter::Trace); // 设置为 Trace 级别，输出所有日志，便于调试
```

同时利用 log crate 提供的函数来按级别过滤日志并进行格式化输出：

```rust
fn enabled(&self, metadata: &Metadata) -> bool {
        metadata.level() <= log::max_level()
}

fn log(&self, record: &Record) {
    // FIXED: Implement the logger with serial output
    if self.enabled(record.metadata()) {
        // 格式化输出：级别 + 目标 + 消息内容
        println!("[{}] {} - {}", record.level(), record.target(), record.args());
    }
}
```

3.3. 运行测试

通过 `make run` 命令运行整个内核，可以看到串口正确初始化，YatSenOS 操作系统被正确加载！

```
linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
[TRACE]: crates/elf/src/lib.rs@071: Map hint: 0xffffff0100000000 -> 0x4397000
[ INFO]: crates/boot/src/fs.rs@063: Free ELF file, pages = 13, addr = 0x00000000043c2000
[ INFO]: crates/boot/src/main.rs@122: Exiting boot services...


__  __       __  _____            ____  _____
\ \/ /___ _/ /_/ ___/___  ____  / __ \/ ___/
 \  / __ `/ __/\__ \/ _ \/ __ \/ / / /\__ \
 / / /_/ / /_ ___/ /  __/ / / / /_/ /___/ /
/_/\__,_/\__//____/\___/_/ /_/\____//____/

                                      v0.1.0
[+] Serial Initialized.
[INFO] ysos_kernel::utils::logger - Logger Initialized.
[INFO] ysos_kernel - YatSenOS initialized.
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
[INFO] ysos_kernel - Hello World from YatSenOS v2!
```

4. 思考题

4.1. 在根目录的 `Cargo.toml` 中，指定了依赖中 `boot` 包为 `default-features = false`，而它会被内核引用，禁用默认 feature 是为了避免什么问题？请结合 `crates/boot` 的 `Cargo.toml` 谈谈你的理解。

尝试删除 `default-features = false` 然后 `make run`，编译会报错，错误信息如下：

```
linkzelda@LAPTOP-QFAEHS2I:~/OS-lab$ make run
   Compiling ysos_kernel v0.1.0 (/home/linkzelda/OS-lab/crates/kernel)
error[E0152]: found duplicate lang item `panic_impl`
   --> crates/kernel/src/utils/macros.rs:69:1
    |
 69 | / fn panic(info: &core::panic::PanicInfo) -> ! {
 70 | |     let stack_trace = collect_stack_trace(MAX_STACK_FRAMES);
...  |
100 | |     loop {}
101 | | }
    | |_^
    |
    = note: the lang item is first defined in crate `uefi` (which `ysos_kernel` depends on)
    = note: first definition in `uefi` loaded from /home/linkzelda/OS-lab/target/x86_64-unknown-no
ne/release/deps/libuefi-cdafeb1ecaaf65d7.rmeta
    = note: second definition in the local crate (`ysos_kernel`)
```

即 `panic_impl` 被重复定义了，观察 `crated/boot` 中的 `Cargo.toml`，其启用了 `uefi/panic_handler` 特性，该特性会提供 `panic_impl`，因此需要禁用其中的默认 feature 才能避免这个问题。

4.2. 在 `crates/boot/src/main.rs` 中参考相关代码，聊聊 `max_phys_addr` 是如何计算的，为什么要这么做?

首先通过 `mmap.entries()` 获取物理内存映射的所有条目，然后通过 `.map()` 传入一个函数计算每个条目的最大地址，随后用 `.max()` 函数计算其中的最大值，最后和 `0x1_0000_0000` 取 `max`。这样的目的在于确保 `max_phy_addr` 确实是内存映射到的最大地址（需要大于所有条目的最大地址，且需要考虑 MMIO 机制可能将 IO 设备映射到了 4GiB(2^32 Bytes) 内的某个地址，因此需要和 `0x1_0000_0000` 取 `max`）

4.3. 串口驱动是在进入内核后启用的，那么在进入内核之前，显示的内容是如何输出的?

在进入内核之前，`bootloader` 运行在 UEFI 环境中，其通过调用 `uefi/logger` 中的接口函数输出日志信息（如使用 `info!` 输出日志和 `log::set_max_level()` 按照级别过滤日志等），这些接口是通过用 UEFI 的文本输出协议在屏幕上打印信息。

4.4. 在 QEMU 中，我们通过指定 -nographic 参数来禁用图形界面，这样 QEMU 会默认将串口输出重定向到主机的标准输出。

4.4.1. 假如我们将 Makefile 中取消该选项，QEMU 的输出窗口会发生什么变化？请观察指令 make run QEMU_OUTPUT= 的输出，结合截图分析对应现象。

取消该选项进行 `make run`，编译不会报错，但在运行时会出现 `gtk initialization failed`，原因在于 WSL 中没有图形界面，无法显示 GUI 窗口。

```
warning: `ysos_kernel` (lib) generated 2 warnings
    Finished `release` profile [optimized] target(s) in 0.05s
cp target/x86_64-unknown-none/release/ysos_kernel esp/KERNEL.ELF
gtk initialization failed
make: *** [Makefile:31: launch] Error 1
```

4.4.2. 在移除 -nographic 的情况下，如何依然将串口重定向到主机的标准输入输出？请尝试自行构造命令行参数，并查阅 QEMU 的文档，进行实验。

可以对 Makefile 进行如下修改：

```
# QEMU_OUTPUT := -nographic
QEMU_OUTPUT := -display none -serial stdio
```

即禁用图形界面输出，并将 serial 串口重定向为标准输入输出，即可以使用 `make run` 命令正常启动内核。

4.5. 结合本实验中 ELF 映射、页表与串口初始化的实现，讨论：

4.5.1. 你会如何划分“最小可启动路径”和“可调试增强路径”？

4.5.2. 若只能保留 3 个关键日志点，你会放在哪些阶段，为什么？

4.6. 若内核映射时误把某段权限设置为可写+可执行，你预期这会对系统安全性和调试复杂度带来什么影响？

4.7. 假设引导阶段必须在“更快启动”和“更多一致性检查”之间二选一，你会如何取舍并给出依据？

5. 加分项

5.1. 线控寄存器的每一比特都有特定的含义，尝试使用 bitflags 宏来定义这些标志位，并在 uart16550 驱动中使用它们。

使用 `bitflags` 宏定义了每个位的名称，然后更改了修改 LCR 线控寄存器的相关代码，修改部分的代码如下

```
bitflags! {
    /// 线控寄存器 (LCR) 的标志位
    pub struct LcrFlags: u8 {
        /// 字长选择位 0 (LSB)
        const WORD_LENGTH_LSB = 1 << 0;
        /// 字长选择位 1 (MSB)
        const WORD_LENGTH_MSB = 1 << 1;
        /// 停止位 (1 表示 2 个停止位, 0 表示 1 个停止位)
        const STOP_BITS       = 1 << 2;
        /// 校验使能
        const PARITY_ENABLE   = 1 << 3;
        /// 偶校验选择 (与 PARITY_ENABLE 配合使用)
        const EVEN_PARITY     = 1 << 4;
        /// 粘性校验
        const STICK_PARITY    = 1 << 5;
        /// 中断控制 (Break)
        const BREAK           = 1 << 6;
        /// 除数锁存器访问位 (DLAB)
        const DLAB            = 1 << 7;
    }
}

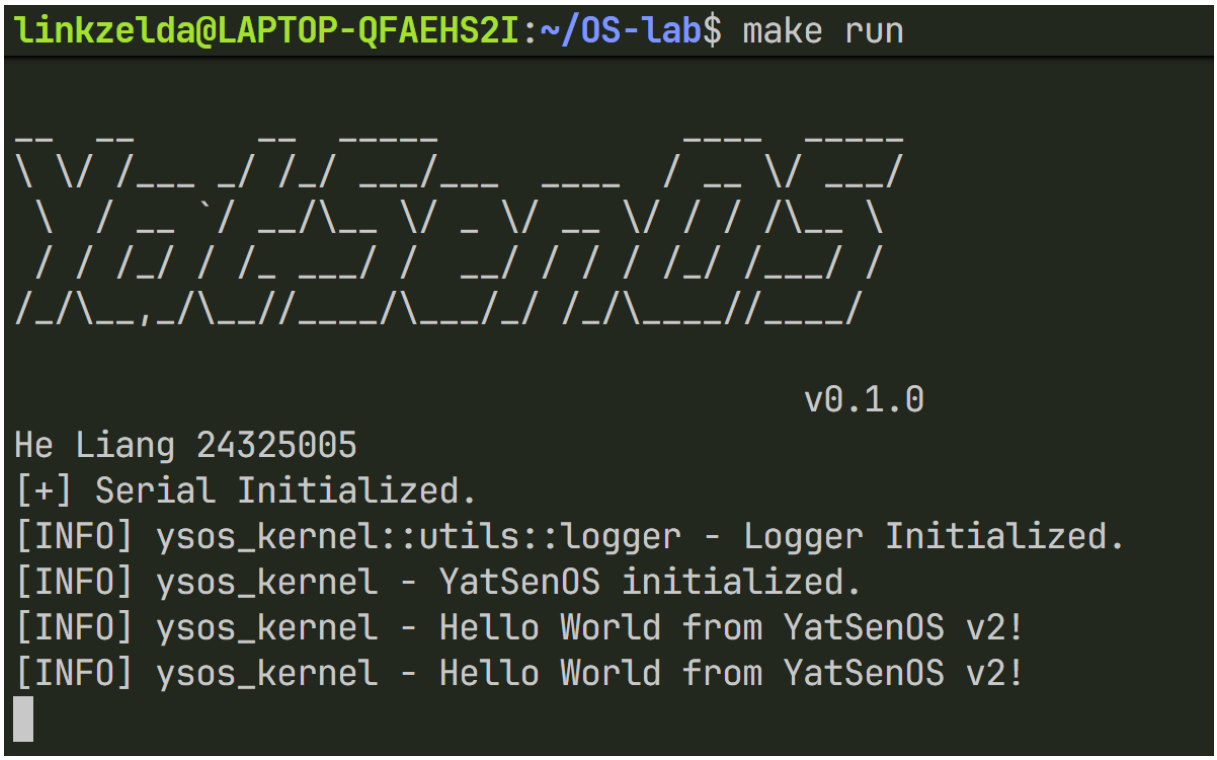
// 设置 DLAB = 1 以访问除数锁存器
Port::new(base + LCR).write(LcrFlags::DLAB.bits());
// 设置数据格式：8 位数据, 无校验, 1 位停止位
Port::new(base + LCR).write((LcrFlags::WORD_LENGTH_LSB |
LcrFlags::WORD_LENGTH_MSB).bits());
```

5.2. 尝试在进入内核并初始化串口驱动后，使用 escape sequence 来清屏，并编辑 get_ascii_header() 中的字符串常量，输出你的学号信息。

在 `serial.rs` 中输出 YSOS 标志的代码前输出一个 escape sequence 清屏。

```
    println!("\x1b[2J\x1b[H"); // 使用 escape sequence 清空
    println!("{}", crate::get_ascii_header());
    println!("[+] Serial Initialized.");
```

然后再在 `get_asciih_header()` 中修改字符串常量，最终输出效果如下：



5.3. 尝试添加字符串型启动配置变量 log_level，并修改 logger 的初始化函数，使得内核能够根据启动参数进行日志输出。

目标是要在 `logger.rs` 中的 `init` 函数中根据 `bootinfo` 中 `log_level` 的值设置 `levelfilter` 的基准值，我们首先在 `config` 类型中加入 `log_level` 参数，对应的调整默认值和输入时的处理，然后构建 `bootinfo` 时添加 `log_level` 参数，并在启动内核时传给 `logger::init()` 内部，在其内部使用 `match` 语句进行处理即可，如下：

```
// FIXED: Configure the logger
    match log_level {
        "Info" => log::set_max_level(log::LevelFilter::Info),
        "Debug" => log::set_max_level(log::LevelFilter::Debug),
        "Error" => log::set_max_level(log::LevelFilter::Error),
        "Off" => log::set_max_level(log::LevelFilter::Off),
        "Trace" => log::set_max_level(log::LevelFilter::Trace),
        "Warn" => log::set_max_level(log::LevelFilter::Warn),
        _ => log::set_max_level(log::LevelFilter::Info)
    }
    // 按照传入的参数设置 levelfilter, 默认为 Info
```

5.4. 尝试使用调试器，在内核初始化之后（ysos::init 调用结束后）下断点，查看、记录并解释如下的信息：

- 内核的栈指针、栈帧指针、指令指针等寄存器的值。

- 内核的代码段、数据段、BSS 段等在内存中的位置。

我们在 `ysos::init` 处设置断点，并开始运行，得到如下输出：

```
[ Legend: Modified register | Code | Heap | Stack | String ]
──────────────────────────────────────────────────────────────────────────────
registers ────
$rax   : 0xffffff00000010d0  →  0xffff27e8e5894855
$rbx   : 0x0000000000200000  →  0x0000000000000000  →  0x0000000000000000  →
[loop detected]
$rcx   : 0x0000000005f0deb0  →  0x00000000059ee018  →  "IBI SYSTF"
$rdx   : 0xffffff01001ffff8  →  0x0000000000000000  →  0x0000000000000000  →
[loop detected]
$rsp   : 0xffffff01001ffeb8  →  0x0000000000000000  →  0x0000000000000000  →
[loop detected]
$rbp   : 0xffffff01001fff38  →  0xffffff01001fffd8  →  0xffffff01001fffe8  →
0x0000000005f0de60  →  0x0000000005f10798  →  0x00000000043a3018  →
0x0000000000000003  →  0x0000000000000000
$rsi   : 0xffff800000000000  →  0x0000000000000000  →  0x0000000000000000  →
[loop detected]
$rdi   : 0x0000000005f0deb0  →  0x00000000059ee018  →  "IBI SYSTF"
$rip   : 0xffffff000000119b  →  0x000009fde83f8b48
$r8    : 0x0000000000000000  →  0x0000000000000000  →  [loop detected]
$r9    : 0x0000000000000501  →  0x0000000000000000  →  0x0000000000000000  →
[loop detected]
$r10   : 0x0000000005ae8860  →  0x7707309600000000
$r11   : 0x0000000000000868  →  0x0000000000000000  →  0x0000000000000000  →
[loop detected]
$r12   : 0x00000000043b9040  →  0x0000000000000003  →  0x0000000000000000  →
0x0000000000000000  →  [loop detected]
$r13   : 0xffffffff1ffaffc0
$r14   : 0x0000000005f10798  →  0x00000000043a3018  →  0x0000000000000003  →
0x0000000000000000  →  0x0000000000000000  →  [loop detected]
$r15   : 0x00000000059ee018  →  "IBI SYSTF"
$eflags: [zero carry PARITY adjust SIGN trap interrupt direction overflow resume
virtualx86 identification]
$cs: 0x38 $ss: 0x30 $ds: 0x30 $es: 0x30 $fs: 0x30 $gs: 0x30
```

可以看到，栈顶寄存器 `$rsp` 的值为 `0xffffff01001ffeb8`，指向当前栈的栈顶；栈帧寄存器 `$rbp` 的值为 `0xffffff01001fff38` 指向当前栈的栈底；PC 寄存器 `$rip` 的值为 `0xffffff000000119b`，指向当前指令的下一条指令。

使用 `info files` 指令可以查看到如下信息：

```
gef➤  info files
Symbols from "/home/linkzelda/OS-lab/esp/KERNEL.ELF".
Remote target using gdb-specific protocol:
        `/home/linkzelda/OS-lab/esp/KERNEL.ELF', file type elf64-x86-64.
        Entry point: 0xffffff00000010d0
        0xffffff0000000000 - 0xffffff0000000684 is .rodata
        0xffffff0000000684 - 0xffffff0000000690 is .eh_frame_hdr
        0xffffff0000000690 - 0xffffff00000006ac is .eh_frame
        0xffffff0000001000 - 0xffffff0000002dfa is .text
        0xffffff0000003000 - 0xffffff00000031f8 is .data
        0xffffff0000004000 - 0xffffff0000004038 is .got
        0xffffff0000005000 - 0xffffff0000005020 is .bss
        While running this, GDB does not access memory from...
Local exec file:
        `/home/linkzelda/OS-lab/esp/KERNEL.ELF', file type elf64-x86-64.
        Entry point: 0xffffff00000010d0
        0xffffff0000000000 - 0xffffff0000000684 is .rodata
        0xffffff0000000684 - 0xffffff0000000690 is .eh_frame_hdr
        0xffffff0000000690 - 0xffffff00000006ac is .eh_frame
        0xffffff0000001000 - 0xffffff0000002dfa is .text
        0xffffff0000003000 - 0xffffff00000031f8 is .data
        0xffffff0000004000 - 0xffffff0000004038 is .got
        0xffffff0000005000 - 0xffffff0000005020 is .bss
```

可以观察到，代码段 `.text` 放置在内存的 `0xffffff0000001000 - 0xffffff0000002dfa` 地址处；数据段 `.data` 放置在内存的 `0xffffff0000003000 - 0xffffff00000031f8` 地址处；`.bss` 段放置在内存的 `0xffffff0000005000 - 0xffffff0000005020` 地址处。